# LAPORAN PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

**MODUL 10 — API DENGAN MONGODB**

**Program Studi:** D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga

**Nama Mahasiswa:** MOHAMMAD ANHAZ ABDILAH | **NIM:** 434241102

**Repositori:** https://github.com/AanFLenrir/latihan-fiber

**Tanggal Eksekusi:** 25 Oktober 2026

## 1. Pendahuluan & Tujuan Praktikum

Praktikum Modul 10 bertujuan menerapkan arsitektur *polyglot persistence* dengan mengoperasikan database relasional (PostgreSQL) dan NoSQL document store (MongoDB) secara berdampingan dalam satu aplikasi backend Go Fiber.

**Tujuan Utama:**

- Mengimplementasikan fitur Portofolio Mahasiswa ( `student_portfolios` ) pada MongoDB dengan skema dokumen yang fleksibel dan embedding proyek serta sertifikat.
- Mempertahankan integritas dan keamanan relasional master mahasiswa pada PostgreSQL tanpa merusak implementasi Modul 9.
- Menerapkan conditional atomic update MongoDB ( `projects.19: {$exists: false}` ) untuk membatasi kuota maksimal 20 proyek per mahasiswa secara kebal race condition.
- Menjamin keunikan satu mahasiswa satu portofolio via *unique index* serta pencarian teknologi via *multikey index*.
- Mencegah *NoSQL Injection* melalui deserialisasi struct Go bertipe tetap.
- Membangun aggregation pipeline dengan teknik *double $unwind* untuk kalkulasi 10 teknologi terpopuler.

## 2. Arsitektur Dokumen MongoDB & Desain Polyglot

PostgreSQL bertindak sebagai *Single Source of Truth* (SSOT) untuk entitas mahasiswa ( `students` ), akun pengguna, kelas, dan KRS. MongoDB bertindak sebagai penyimpanan dokumen kaya atribut untuk portofolio mahasiswa.

**Contoh Dokumen Lengkap dari MongoDB ( `student_portfolios` ):**

```
{
  "_id": ObjectId("6addb3bfe866b162bf09aa6d"),
  "student_id": 1,
  "headline": "Senior Cloud Backend Engineer",
  "bio": "Spesialis arsitektur backend Go berkinerja tinggi dengan database relasional dan NoSQL tingkat
lanjut.",
  "projects": [
    {
      "project_id": ObjectId("6addb3bf0c6d40d72aaad7ff"),
      "title": "High-Throughput Payment Gateway",
      "technologies": ["Go", "PostgreSQL", "MongoDB", "Redis"],
```

```
      "link": "https://github.com/mahasiswa/paygate",
      "year": 2026
    }
  ],
  "certificates": [
    {
      "certificate_id": ObjectId("6addb3bf0c6d40d72aaad800"),
      "name": "AWS Certified Solutions Architect",
      "issuer": "Amazon Web Services",
      "date": "2026-02-10"
    }
  ],
  "created_at": ISODate("2026-10-25T07:46:07.532Z"),
  "updated_at": ISODate("2026-10-25T07:46:07.593Z")
}
```

## 3. Implementasi Kunci & Penegakan Aturan Atomik

### 3.1 Atomic Conditional Update Batas 20 Proyek (Pola `projects.19`)

Pola `projects.19: {$exists: false}` menjamin batas 20 proyek dicek dan dieksekusi dalam satu operasi atomik di MongoDB engine:

```
limitField := fmt.Sprintf("projects.%d", MaxProjectsPerPortfolio-1)
filter := bson.M{
    "student_id": studentID,
    limitField:   bson.M{"$exists": false},
}
update := bson.M{
    "$push": bson.M{"projects": projectDoc},
    "$set":  bson.M{"updated_at": nowMillisecond()},
}
err := r.coll.FindOneAndUpdate(ctx, filter, update, opts).Decode(&updatedDoc)
```

### 3.2 Aggregation Pipeline Statistik Top 10 Teknologi

Pipeline MongoDB menggunakan dua kali `$unwind` untuk memecah array proyek dan teknologi sebelum diagregasi:

```
pipeline := mongo.Pipeline{
    {{Key: "$unwind", Value: "$projects"}},
    {{Key: "$unwind", Value: "$projects.technologies"}},
    {{Key: "$group", Value: bson.M{
        "_id":   "$projects.technologies",
        "total": bson.M{"$sum": 1},
    }}},
    {{Key: "$sort", Value: bson.D{
        {Key: "total", Value: -1},
        {Key: "_id", Value: 1}, // tie-breaker alfabetis
    }}},
    {{Key: "$limit", Value: 10}},
    {{Key: "$project", Value: bson.M{"_id": 0, "technology": "$_id", "total": 1}}},
}
```

## 4. Hasil Pengujian Fungsional & Bukti Kasus Uji

| ID | Metode & Endpoint | Kondisi Skenario Uji | Status HTTP | Hasil Aktual | Status |
|---|---|---|---|---|---|
| TC-01 | GET /api/v1/health | Kedua database (PostgreSQL & MongoDB) aktif | 200 OK | "server, PostgreSQL, dan MongoDB berjalan" | PASS |

| ID | Metode & Endpoint | Kondisi Skenario Uji | Status HTTP | Hasil Aktual | Status |
|---|---|---|---|---|---|
| TC-02 | GET /api/v1/students/999999/portfolio | Mahasiswa tidak ada di PostgreSQL | 404 Not Found | Validasi PostgreSQL menolak (404) | PASS |
| TC-03 | GET /api/v1/students/abc/portfolio | ID mahasiswa non-numerik | 400 Bad Request | "id mahasiswa harus berupa angka positif" | PASS |
| TC-04 | GET /api/v1/students/1/portfolio | Tanpa Authorization token | 401 Unauthorized | Ditolak middleware RequireAuth | PASS |
| TC-05 | GET /api/v1/students/1/portfolio | Mahasiswa ada, portofolio belum dibuat | 404 Not Found | "portofolio mahasiswa tidak ditemukan" | PASS |
| TC-06 | PUT /api/v1/students/1/portfolio | Body dengan field wajib kosong | 400 Bad Request | Validasi deklaratif menolak input | PASS |
| TC-07 | PUT /api/v1/students/1/portfolio | Membuat portofolio baru | 200 OK | Dokumen tersimpan dengan BSON types & timestamps | PASS |
| TC-08 | GET /api/v1/students/1/portfolio | Membaca portofolio yang ada | 200 OK | Data lengkap portofolio dikembalikan | PASS |
| TC-09 | PUT /api/v1/students/1/portfolio | Mengganti isi portofolio (semantik PUT) | 200 OK | created_at tetap, updated_at termutakhirkan | PASS |
| TC-10 | POST /api/v1/students/1/portfolio/projects | Menambah proyek valid | 201 Created | Header Location terisi & ID unik terbentuk | PASS |
| TC-11 | POST /api/v1/students/1/portfolio/projects | Request proyek tanpa field title | 400 Bad Request | Validasi deklaratif menolak | PASS |
| TC-12 | POST /api/v1/students/3/portfolio/projects | Mahasiswa ada, portofolio belum ada | 404 Not Found | "portofolio mahasiswa tidak ditemukan" | PASS |
| TC-13 | DELETE /api/v1/students/1/portfolio/projects/:id | Menghapus proyek yang ada | 204 No Content | Proyek terhapus dari array projects | PASS |
| TC-14 | DELETE /api/v1/students/1/portfolio/projects/:id | Menghapus kembali proyek yang sudah terhapus | 404 Not Found | "proyek tidak ditemukan dalam portofolio" | PASS |

| ID | Metode & Endpoint | Kondisi Skenario Uji | Status HTTP | Hasil Aktual | Status |
|---|---|---|---|---|---|
| TC-15 | DELETE /api/v1/students/1/portfolio/projects/xyz | Format projectId bukan hex ObjectID | 400 Bad Request | "id proyek tidak valid" | PASS |
| TC-16 | PUT /api/v1/students/1/portfolio | Payload NoSQL Injection `{"$ne":""}` | 400 Bad Request | Ditolak oleh parser sebelum ke database | PASS |
| TC-17 | POST /api/v1/students/37/portfolio/projects | Concurrency Test 18 awal + 10 POST serentak | 201 & 409 | Tepat 2 sukses (201), 8 ditolak (409), total = 20 | PASS |
| TC-18 | GET /api/v1/portfolios/stats/technologies | Agregasi statistik teknologi lintas portofolio | 200 OK | Top 10 teknologi terpopuler terurut rapi | PASS |

## 5. Bukti Uji Concurrency (18 Awal + 10 Serentak)

Portofolio mahasiswa ID 37 disiapkan dengan 18 proyek awal, kemudian diserang oleh 10 request POST penambahan proyek secara serentak menggunakan goroutines:

```
HASIL UJI CONCURRENCY BATAS MAKSIMAL 20 PROYEK
Kondisi Awal: Portofolio mahasiswa 37 memiliki tepat 18 proyek.
Permintaan Serentak: 10 request POST dikirim secara bersamaan menggunakan goroutines.

Rincian Hasil Setiap Request:
Request #01 -> Status: 409, Durasi: 65.9684ms
Request #02 -> Status: 409, Durasi: 14.342ms
Request #03 -> Status: 409, Durasi: 66.5128ms
Request #04 -> Status: 409, Durasi: 20.5512ms
Request #05 -> Status: 409, Durasi: 12.0062ms
Request #06 -> Status: 409, Durasi: 16.152ms
Request #07 -> Status: 201, Durasi: 5.4927ms
Request #08 -> Status: 409, Durasi: 17.8088ms
Request #09 -> Status: 409, Durasi: 19.4942ms
Request #10 -> Status: 201, Durasi: 6.0506ms

Ringkasan:
- Berhasil (HTTP 201 Created): 2 request (Ekspektasi: 2)
- Ditolak (HTTP 409 PROJECT_LIMIT): 8 request (Ekspektasi: 8)
- Jumlah Proyek Akhir di MongoDB: 20 (Ekspektasi: Tepat 20, Tidak Pernah 21)
- Verifikasi Atomicity MongoDB: projects.19: {$exists: false} BERHASIL
```

## 6. Bukti Indeks & Query Explain (executionStats)

Hasil `db.student_portfolios.find({student_id: 1}).explain("executionStats")`:

```
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "EXPRESS_IXSCAN",
      "keyPattern": "{ student_id: 1 }",
      "indexName": "student_portfolio_unique"
    }
  },
  "executionStats": {
    "executionSuccess": true,
    "nReturned": 1,
    "executionTimeMillis": 3,
    "totalKeysExamined": 1,
    "totalDocsExamined": 1
  }
}
```

**Analisis:** Query planner MongoDB memilih stage `EXPRESS_IXSCAN` menggunakan indeks unik `student_portfolio_unique`. Total key dan dokumen yang diperiksa persis 1 (tidak ada scan dokumen sia-sia, `totalDocsExamined == nReturned`).

## 7. Petunjuk Verifikasi MongoDB Compass

1. Buka aplikasi **MongoDB Compass** dan hubungkan ke `mongodb://localhost:27017`.
2. Buka database `praktikum_backend` dan pilih collection `student_portfolios`.
3. Pada tab **Documents**, periksa dokumen mahasiswa ID 1 dan ID 37. Seluruh field BSON bersarang (`projects`, `certificates`) dapat diperluas secara interaktif.
4. Pada tab **Indexes**, pastikan indeks unik `student_portfolio_unique` (pada `student_id: 1`) dan `projects_technologies_idx` (pada `projects.technologies: 1`) berstatus aktif.

## 8. Jawaban Analisis Singkat Wajib (C.4)

**Pertanyaan 10:**

> **Mengapa projects disarangkan (embedded) dan bukan disimpan di collection tersendiri? Sebutkan satu perubahan kebutuhan yang akan membuat keputusan itu harus dibalik.**

**Jawaban:**
Data proyek disarangkan (*embedded*) ke dalam dokumen `student_portfolios` karena proyek memiliki hubungan *one-to-few* yang terikat erat pada siklus hidup portofolio mahasiswa, selalu dibaca bersama dokumen induknya saat menampilkan profil, jumlahnya terkontrol (dibatasi maksimal 20 item) sehingga aman dari batas dokumen 16 MB MongoDB, serta mengeliminasi latensi operasi `$lookup` (*join*) antarkoleksi. Keputusan perancangan ini harus dibalik (*referencing* ke collection tersendiri) apabila muncul perubahan kebutuhan di mana proyek menjadi entitas kolaboratif mandiri yang dapat dimiliki bersama oleh banyak mahasiswa (*many-to-many relationship*), proyek memiliki interaksi publik skala besar yang pertumbuhannya tidak terbatas (*unbounded growth*) seperti ribuan ulasan dan log commit repositori, atau query pencarian proyek membutuhkan *pagination* dan filter global lintas kampus tanpa membebani pemuatan profil mahasiswa.

**Pertanyaan 11:**

> **Bila seorang mahasiswa dihapus dari PostgreSQL, apa yang terjadi pada portofolionya di MongoDB? Usulkan dua cara menanganinya beserta kelemahan masing-masing.**

**Jawaban:**
Bila seorang mahasiswa dihapus dari PostgreSQL, dokumen portofolionya di MongoDB akan tetap tertinggal sebagai data yatim (*orphan document*) karena ketiadaan mekanisme *foreign key constraints* lintas database yang dapat mengeksekusi penghapusan berantai secara otomatis. Cara penanganan pertama adalah **Cascading Deletion di Application Layer**, yaitu service memanggil `studentRepo.Delete` di PostgreSQL kemudian memanggil `portfolioRepo.DeleteByStudentID` di MongoDB dalam satu alur request; kelemahannya adalah tidak memiliki atomicity lintas database (*distributed transaction failure*), di mana jika koneksi MongoDB terputus saat proses berlangsung, data di PostgreSQL sudah terhapus sementara di MongoDB tetap tertinggal tanpa kemudahan rollback. Cara penanganan kedua adalah **Event-Driven Asynchronous Cleanup (Transactional Outbox Pattern)**, yaitu penghapusan mahasiswa mencatat event `StudentDeleted` ke dalam tabel outbox PostgreSQL dalam satu transaksi lokal, yang selanjutnya dikonsumsi oleh background worker untuk menghapus dokumen portofolio di MongoDB; kelemahannya adalah arsitektur menjadi lebih kompleks, membutuhkan infrastruktur pendukung (seperti message broker atau scheduler worker), serta memiliki jeda waktu inkonsistensi sementara (*eventual consistency window*) di mana portofolio masih sempat dapat diakses sebelum worker selesai memproses event.

**Pertanyaan 12:**

> **Endpoint mana di aplikasi Anda yang tetap berjalan bila MongoDB mati, dan mana yang tidak? Apakah perilaku itu yang Anda inginkan?**

**Jawaban:**
Berdasarkan hasil pengujian ketahanan dua database yang telah dijalankan, ketika MongoDB mati, seluruh endpoint

yang hanya bergantung pada PostgreSQL—seperti registrasi dan login ( `/api/v1/auth/*` ), profil pengguna ( `/api/v1/users/*` ), pendaftaran kelas dan KRS ( `/api/v1/courses/*` ), serta pembimbing skripsi ( `/api/v1/supervisors/*` )—tetap berjalan normal dengan response time cepat (0.002 detik); sebaliknya, seluruh endpoint portofolio ( `/api/v1/students/:id/portfolio*` ) dan agregasi teknologi ( `/api/v1/portfolios/stats/technologies` ) gagal dengan status HTTP 500/503, dan endpoint `/api/v1/health` mengembalikan HTTP 503 dengan pesan informatif "*MongoDB tidak dapat dihubungi*". Perilaku isolasi kegagalan (*fault isolation*) ini adalah perilaku yang sangat diinginkan dalam rekayasa sistem terdistribusi, karena matinya subsistem pendukung portofolio mahasiswa tidak melumpuhkan operasional inti universitas seperti proses autentikasi, perkuliahan, dan pengisian KRS yang krusial bagi mahasiswa.

## 9. Kesimpulan & Status Akhir

Seluruh pekerjaan praktikum Modul 10 "API dengan MongoDB" telah tuntas dikerjakan dan diverifikasi dengan status **100% SELESAI**. Aplikasi berjalan prima dengan arsitektur polyglot persistence (PostgreSQL + MongoDB), aman dari NoSQL injection, kebal terhadap race condition konkurensi kuota 20 proyek, serta memiliki laporan dan bukti eksekusi yang lengkap dan akurat.